MANUAL DE USUARIO

PokéWiki — Enciclopedia Pokémon

Guía práctica de HTML, CSS, JavaScript y PokéAPI

Especialidad de programación

Dra. Mayleth Sánchez Pérez

Semestre: 5°

Turno: Vespertino

Índice

1. Introducción

2. ¿Qué es PokéWiki?

3. Tecnologías utilizadas

4. Cómo crear el proyecto desde cero

5. Estructura general del archivo HTML

6. Explicación de la sección HTML

7. Explicación del CSS

8. Explicación del JavaScript

9. Cómo funciona PokéAPI

10. Funcionamiento de la búsqueda, filtros y generaciones

11. Paginación

12. Ventana de información del Pokémon

13. Formas, shiny y evoluciones

14. Cómo modificar el proyecto

15. Cómo ejecutar y probar la página

16. Errores frecuentes y soluciones

17. Recomendaciones para mejorar el proyecto

18. Resumen del funcionamiento

1. Introducción

Este manual explica cómo crear, ejecutar, utilizar y modificar el proyecto web PokéWiki. El proyecto es una enciclopedia Pokémon desarrollada con HTML, CSS y JavaScript. La información de los Pokémon se obtiene dinámicamente desde PokéAPI, por lo que el contenido no está escrito manualmente en cada tarjeta.

La finalidad de esta guía es que una persona que conozca los conceptos básicos de programación web pueda comprender qué hace cada parte del código y pueda modificarlo sin tener que rehacer todo el proyecto.

2. ¿Qué es PokéWiki?

PokéWiki es una página web educativa que presenta una Pokédex. Al cargarla, el JavaScript consulta PokéAPI y obtiene una lista de Pokémon. Después muestra las especies mediante tarjetas y permite consultar información adicional.

- Lista de Pokémon y numeración de Pokédex.
- Búsqueda por nombre o número.
- Filtro por tipo.
- Ordenamiento por número o nombre.
- Filtro por generaciones I a IX.
- Paginación de resultados.
- Ventana modal con información detallada.
- Estadísticas base.
- Imagen normal y variante shiny.
- Formas y transformaciones disponibles en PokéAPI.
- Cadena evolutiva.
- Diseño adaptable a diferentes tamaños de pantalla.

3. Tecnologías utilizadas

Tecnología	Uso	Dónde aparece
HTML5	Construye la estructura y el contenido.	<html>, <head>, <body>, <header>, <main>, <aside>, <footer>
CSS3	Controla colores, tamaños, posiciones, tarjetas, modal y diseño responsive.	<style>
JavaScript	Consulta la API, crea tarjetas, filtros, eventos, modal y paginación.	<script>
PokéAPI	Proporciona los datos de Pokémon.	https://pokeapi.co/api/v2
Fetch API	Realiza solicitudes HTTP desde JavaScript.	fetch(...)

4. Cómo crear el proyecto desde cero

La forma más sencilla es utilizar un solo archivo HTML.

1. Crea una carpeta llamada, por ejemplo, PokéWiki.
2. Dentro de la carpeta crea un archivo llamado index.html.
3. Abre index.html con Visual Studio Code, Notepad++ u otro editor.
4. Copia el código HTML, CSS y JavaScript dentro del archivo.
5. Guarda el archivo.
6. Abre index.html en un navegador moderno.
7. Asegúrate de tener conexión a Internet porque los datos y las imágenes se consultan desde servicios externos.

Estructura mínima:

```
PokéWiki/
└── index.html
```

También puedes separar el proyecto en tres archivos (index.html, style.css y script.js), pero el código proporcionado ya está preparado como un documento único.

5. Estructura general del archivo HTML

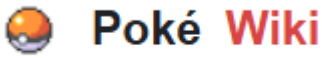
```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>PokéWiki — Enciclopedia Pokémon</title>
    <style>
        /* CSS */
    </style>
</head>
<body>
    <!-- Contenido de la página -->

    <script>
        // JavaScript
    </script>
</body>
</html>
```

El documento se divide en tres grandes partes: HTML para la estructura, CSS para la apariencia y JavaScript para la interacción y los datos.

Sección	Función
<head>	Configuración, metadatos, título y CSS.
<body>	Contenido visible de la página.
<style>	Reglas visuales CSS.
<script>	Lógica JavaScript.

6. Explicación de la sección HTML



6.1 Encabezado

El <header> contiene el logotipo y el nombre de la enciclopedia. La imagen de la Poké Ball se obtiene mediante una URL externa.

```
<header>
  <div class="logo">
    <img src="URL_DE_LA_IMAGEN" alt="Poké Ball">
    Poké<span>Wiki</span>
  </div>
</header>
```

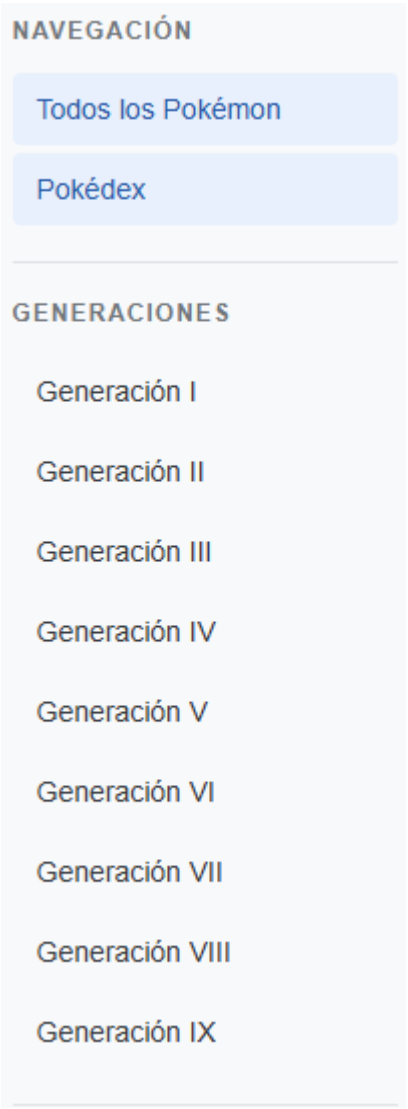
La clase logo permite controlar mediante CSS la posición, tamaño y tipografía.

6.2 Barra lateral

El <aside> contiene los controles de navegación. Incluye el enlace de todos los Pokémon y enlaces para seleccionar las generaciones I a IX.

```
<a class="sidebar-link generation-link"
   data-generation="1" href="#">
   Generación I
</a>
```

El atributo data-generation guarda el número de generación. JavaScript lo lee con link.dataset.generation.



6.3 Área principal

Dentro de <main> se encuentra el título, la descripción, el buscador, los filtros, la cuadrícula de Pokémon y la paginación.

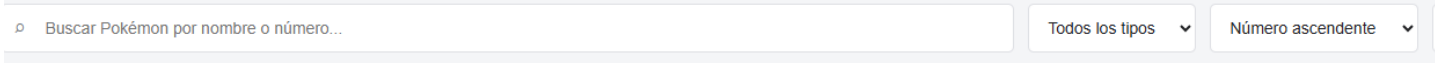
6.4 Barra de herramientas

El buscador utiliza un <input>. Los filtros utilizan <select> y el botón de limpieza restablece los valores.

```
<input id="search"
       type="text"
       placeholder="Buscar Pokémon por nombre o número...">

<select id="type-filter">
    <option value="">Todos los tipos</option>
</select>

<button id="reset-filters">
    Limpiar filtros
</button>
```

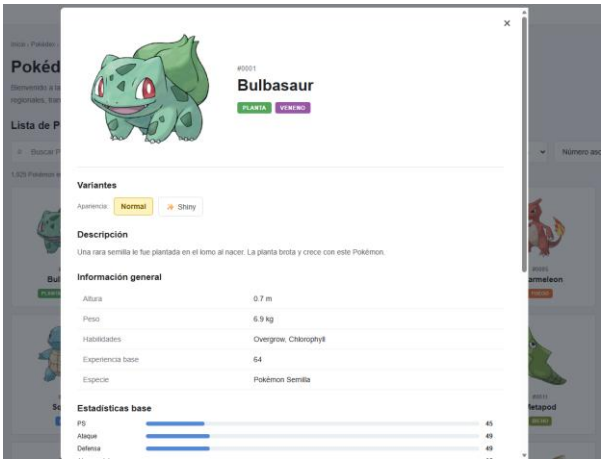


6.5 Cuadrícula

El contenedor pokemon-grid comienza con un mensaje de carga. JavaScript elimina ese mensaje y crea dinámicamente las tarjetas mediante createPokemonCard().

6.6 Modal

El modal es una ventana superpuesta que permanece oculta hasta que el usuario selecciona un Pokémon. La clase show hace que se muestre.



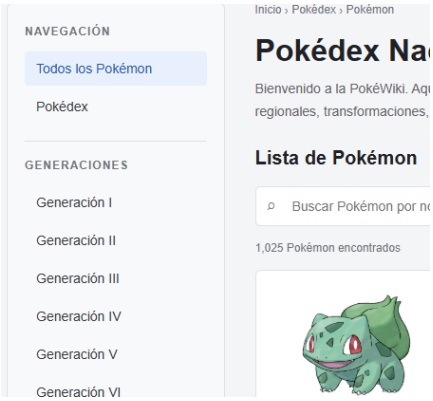

```
<div class="modal" id="pokemon-modal">
    <div class="modal-content">
        ...
    </div>
</div>
```

7. Explicación del CSS

El CSS define la apariencia completa del proyecto. Está organizado por secciones, lo que facilita encontrar una regla cuando se desea modificarla.

7.1 Variables de color

```
:root {
  --bg: #f4f5f7;
  --white: #ffffff;
  --text: #202124;
  --muted: #70757a;
  --border: #dfe1e5;
  --blue: #2457a7;
}
```



Las variables permiten cambiar colores desde un solo lugar. Por ejemplo, cambiar --blue modifica el color principal utilizado por diferentes elementos.

7.2 Layout principal

La clase .layout utiliza CSS Grid para separar la barra lateral y el contenido principal.

```
.layout {
  display: grid;
  grid-template-columns: 230px minmax(0, 1fr);
  gap: 32px;
}
```

7.3 Tarjetas de Pokémon

pokemon-grid utiliza una cuadrícula. Cada pokemon-card tiene borde, fondo blanco, bordes redondeados y un pequeño movimiento cuando el cursor pasa por encima.




```
.pokemon-grid {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: 14px;
}
```

7.4 Tipos

Cada tipo tiene una clase CSS independiente, como .fire, .water, .grass o .dragon. JavaScript genera la clase correspondiente al leer el tipo desde PokéAPI.

7.5 Diseño responsive

Las reglas @media modifican la cuadrícula cuando la ventana es más pequeña. En pantallas grandes se muestran más tarjetas por fila y en pantallas más pequeñas se reduce la cantidad.

8. Explicación del JavaScript

JavaScript es la parte que hace interactiva la aplicación. Consulta la API, guarda los datos, filtra resultados, genera HTML y responde a las acciones del usuario.

8.1 Configuración

```
const API = "https://pokeapi.co/api/v2";
const PAGE_SIZE = 30;

let allPokemon = [];
let filteredPokemon = [];
let currentPage = 1;
let currentGeneration = 0;
```

allPokemon contiene la lista nacional. filteredPokemon contiene los resultados actuales. currentPage controla la página mostrada y currentGeneration guarda la generación seleccionada.

8.2 Obtener elementos HTML

```
const grid = document.getElementById("pokemon-grid");
const searchInput = document.getElementById("search");
const typeFilter = document.getElementById("type-filter");
```

document.getElementById() permite que JavaScript encuentre elementos del HTML mediante su id.

8.3 Funciones auxiliares

Función	Propósito
capitalize()	Pone en mayúscula la primera letra.
formatNumber()	Convierte un número en formato #0001, #0025, etc.
getPokemonId()	Obtiene el ID numérico desde una URL.
cleanName()	Reemplaza guiones y capitaliza palabras.
getTypeHTML()	Crea el HTML de una etiqueta de tipo.
showGridMessage()	Muestra mensajes de carga o error.

8.4 Cargar la lista nacional

```
const response = await fetch(
  `${API}/pokemon?limit=1025&offset=0`
);
```

fetch() realiza una petición a PokéAPI. await espera la respuesta. Después response.json() convierte la respuesta a un objeto JavaScript.

El resultado contiene nombres y URLs. El código transforma esos resultados en objetos con name, id y url y los almacena en allPokemon.

8.5 Filtrar y ordenar

applyFilters() lee el texto del buscador, el tipo seleccionado y el criterio de ordenamiento. Primero filtra por texto y generación y posteriormente ordena los resultados.

```
const matchesSearch =
  pokemon.name.toLowerCase().includes(search) ||
  String(pokemon.id).includes(search);
```

Esto permite encontrar un Pokémon escribiendo parte de su nombre o su número.

8.6 Filtrar por tipo

Cuando se selecciona un tipo, filterByType() consulta el endpoint correspondiente, por ejemplo /type/fire. Después crea un Set con los IDs de los Pokémon de ese tipo y conserva únicamente los resultados coincidentes.

8.7 Crear una tarjeta

createPokemonCard(pokemon) crea un elemento article, coloca la imagen, el número, el nombre y los tipos, y finalmente registra un evento click para abrir el modal.

```
card.addEventListener("click", () => {
  openPokemonModal(pokemon.id);
});
```

9. Cómo funciona PokéAPI

PokéAPI es una API pública que entrega información estructurada de Pokémon mediante URLs. El proyecto no almacena toda la información localmente; la solicita cuando la necesita.

Endpoint	Información
/pokemon?limit=1025&offset=0	Lista nacional de Pokémon.
/pokemon/{id}	Datos de un Pokémon concreto.
/pokemon-species/{id}	Especie, descripciones, variedades y evolución.
/type/{tipo}	Pokémon pertenecientes a un tipo.
evolution_chain.url	Cadena de evolución relacionada con una especie.

La aplicación necesita Internet para consultar estos endpoints. Si la API no responde, la página puede mostrar mensajes de error.

10. Búsqueda, filtros y generaciones

10.1 Búsqueda

El evento input se ejecuta cada vez que el usuario escribe. El programa vuelve a aplicar los filtros y actualiza la lista.

```
searchInput.addEventListener("input", applyFilters);
```

10.2 Filtro de tipo

El select contiene los tipos disponibles. Al cambiar su valor se ejecuta applyFilters(). Si existe un tipo seleccionado, se realiza una consulta adicional a PokéAPI.

10.3 Generaciones

```
const generationRanges = {
  1: [1, 151],
  2: [152, 251],
  3: [252, 386],
  4: [387, 493],
  5: [494, 649],
  6: [650, 721],
  7: [722, 809],
  8: [810, 905],
  9: [906, 1025]
};
```

El filtro de generación compara el ID del Pokémon con el rango correspondiente. Por ejemplo, la Generación I utiliza IDs del 1 al 151.

11. Paginación

La constante PAGE_SIZE está establecida en 30, por lo que se muestran hasta 30 Pokémon por página.

```
const start = (currentPage - 1) * PAGE_SIZE;

const pagePokemon = filteredPokemon.slice(
  start,
  start + PAGE_SIZE
);
```

slice() obtiene únicamente los elementos correspondientes a la página actual. Los botones Anterior y Siguiente cambian currentPage y vuelven a renderizar la página.

12. Ventana de información del Pokémon

Al seleccionar una tarjeta se ejecuta openPokemonModal(id). La función solicita dos recursos simultáneamente: los datos generales del Pokémon y los datos de su especie.

```
const [pokemonResponse, speciesResponse] =
  await Promise.all([
    fetch(`${API}/pokemon/${id}`),
    fetch(`${API}/pokemon-species/${id}`)
  ]);
```

Promise.all() permite esperar las dos solicitudes antes de continuar. Esto proporciona la información necesaria para construir el modal.

El modal muestra:

- Número y nombre.
- Tipos.
- Imagen.
- Descripción en español, cuando está disponible.
- Altura y peso.
- Habilidades.
- Experiencia base.

- Especie.
- Estadísticas.
- Formas.
- Cadena evolutiva.

13. Formas, shiny y evoluciones

13.1 Pokémon shiny

La API proporciona imágenes normales y shiny. El botón Normal coloca la imagen estándar y el botón Shiny cambia a front_shiny.

13.2 Formas

loadPokemonForms() revisa species.varieties y consulta cada variante. createFormCard() genera una tarjeta para cada forma.

getFormCategory() intenta identificar categorías mediante el nombre, incluyendo Mega Evolución, Gigamax y formas regionales de Alola, Galar, Hisui y Paldea.

13.3 Evoluciones

getEvolutionChain() consulta la cadena de evolución y recorre los nodos mediante la función walk(). renderEvolutionChain() convierte los IDs obtenidos en tarjetas visuales.

Las tarjetas de evolución tienen un evento click, por lo que el usuario puede abrir directamente la información del Pokémon correspondiente.

14. Cómo modificar el proyecto

14.1 Cambiar el título

```
<title>Mi Pokédex</title>
```

14.2 Cambiar el nombre mostrado

```
Poké<span>Wiki</span>
```

14.3 Cambiar colores

```
:root {
  --blue: #2457a7;
  --bg: #f4f5f7;
  --text: #202124;
}
```

14.4 Cambiar cantidad por página

```
const PAGE_SIZE = 50;
```

Con esto se intentarán mostrar hasta 50 resultados por página.

14.5 Cambiar cantidad de Pokémon cargados

```
`${API}/pokemon?limit=1025&offset=0`
```

El parámetro limit determina cuántos resultados de la lista se solicitan. Si se reduce, la Pokédex tendrá menos Pokémon disponibles.

14.6 Agregar un nuevo tipo

Se debe añadir una opción al select, una entrada al objeto typeNames y una clase CSS para el color del tipo. Esto solo tiene sentido si el tipo también existe en la API.

14.7 Cambiar el texto descriptivo

Los textos visibles de HTML pueden modificarse directamente. La información dinámica, en cambio, proviene de PokéAPI y normalmente debe transformarse desde JavaScript.

15. Cómo ejecutar y probar la página

Para una prueba básica, el archivo puede abrirse directamente en el navegador. Para un desarrollo más cómodo se recomienda utilizar un servidor local, por ejemplo la extensión Live Server de Visual Studio Code.

8. Abre la carpeta del proyecto en Visual Studio Code.
9. Abre index.html.
10. Inicia Live Server si está instalado.
11. Abre la dirección local proporcionada por el editor.
12. Comprueba que aparezcan las tarjetas.
13. Prueba el buscador.
14. Prueba cada tipo.
15. Prueba las generaciones.
16. Abre una tarjeta y revisa el modal.
17. Prueba Normal/Shiny, formas y evoluciones.
18. Prueba los botones de paginación.

Si la lista no aparece, abre las herramientas de desarrollador del navegador con F12 y revisa la pestaña Console para localizar errores.

16. Errores frecuentes y soluciones

Problema	Posible causa	Solución
No aparecen Pokémon	No hay conexión o la API no responde.	Comprueba Internet y revisa Console.
Imágenes vacías	La URL de imagen no está disponible.	Comprueba sprites y usa el respaldo de front_default.
El filtro no funciona	El id del elemento no coincide con JavaScript.	Verifica que HTML y getElementById() usen el mismo id.
El modal no abre	El evento click no se registró.	Comprueba createPokemonCard() y openPokemonModal().
Error de sintaxis	Se modificó una llave, paréntesis, comilla o plantilla.	Revisa Console y la línea indicada.

No carga en un navegador	Se pegó el código con caracteres de escape o formato incorrecto.	Usa HTML real: <, >, comillas normales y backticks donde corresponda.

Importante: el texto proporcionado en la consulta contiene caracteres de escape como \<, \> y algunas secuencias que parecen haberse agregado al copiar el código. En un archivo HTML real deben utilizarse los caracteres normales. Por ejemplo, debe escribirse <html> y no \<html>.

17. Recomendaciones para mejorar el proyecto

- Separar HTML, CSS y JavaScript en archivos independientes cuando el proyecto crezca.
- Agregar una pantalla de carga más clara.
- Controlar mejor los errores de red.
- Evitar solicitudes repetidas mediante caché de datos.
- Agregar un filtro por habilidades o estadísticas.
- Agregar información de movimientos.
- Agregar favoritos usando localStorage.
- Agregar modo oscuro.
- Mejorar la accesibilidad con etiquetas y estados ARIA.
- Hacer que el diseño sea completamente adaptable a teléfonos móviles.
- Usar un servidor local durante el desarrollo.
- Mantener las URLs de la API y otros valores configurables en un solo lugar.

18. Resumen del funcionamiento

El funcionamiento general puede resumirse en el siguiente flujo:

19. El navegador carga index.html.
20. HTML construye la estructura de la Pokédex.
21. CSS aplica el diseño visual.
22. JavaScript obtiene referencias a los elementos HTML.
23. loadPokemonList() consulta la lista de Pokémon en PokéAPI.
24. Los resultados se guardan en allPokemon.
25. applyFilters() determina qué Pokémon deben mostrarse.
26. renderCurrentPage() calcula la página y solicita los detalles necesarios.
27. createPokemonCard() crea cada tarjeta.
28. El usuario puede buscar, ordenar, filtrar o cambiar de generación.
29. Al hacer clic en una tarjeta, openPokemonModal() consulta información detallada.
30. El modal muestra datos, estadísticas, formas, shiny y evoluciones.
31. Los eventos de botones y teclado permiten cerrar el modal y navegar.

Conclusión

PokéWiki es un ejemplo completo de una aplicación web dinámica de una sola página. HTML proporciona la estructura, CSS se encarga de la presentación y JavaScript conecta la interfaz con PokéAPI. Comprender estas tres partes permite modificar el proyecto, agregar nuevas funciones y utilizar la misma arquitectura para otros tipos de catálogos o enciclopedias que utilicen una API.